

DS Engineer Test Task

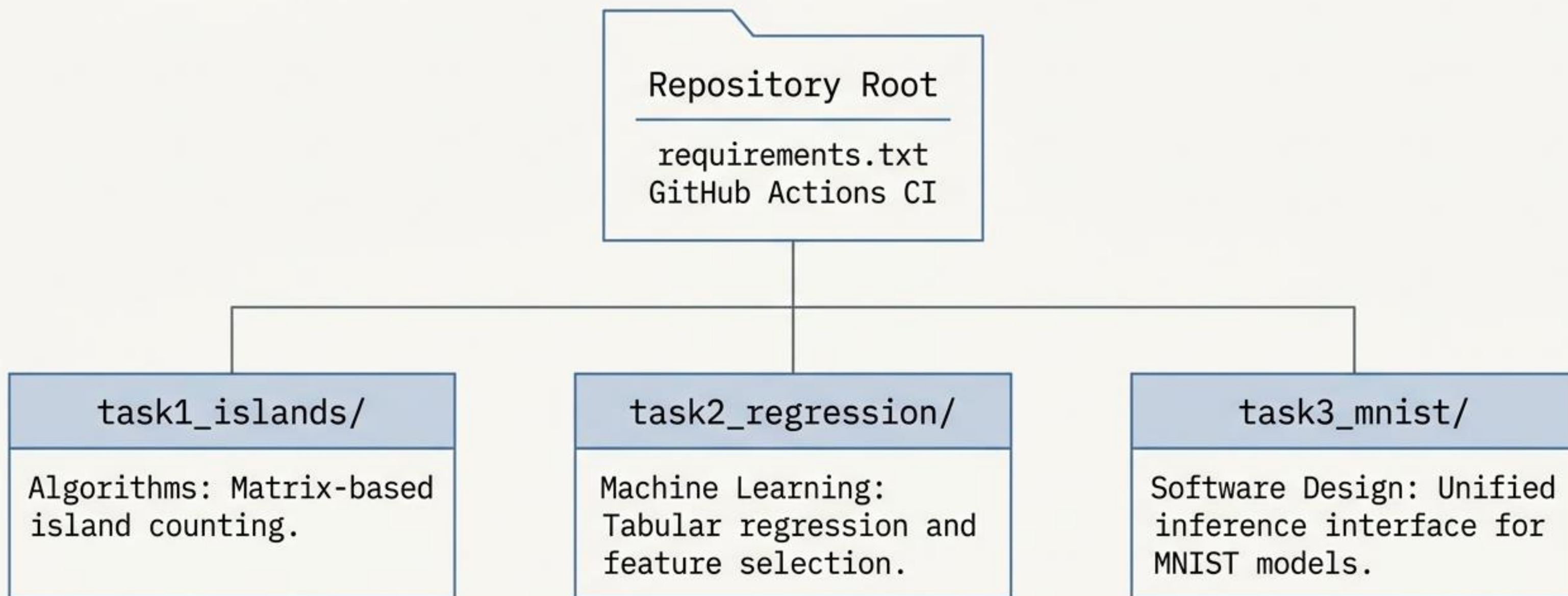
Algorithms, Tabular Regression,
and MNIST Inference Design

Project Metadata

Repository:
[https://github.com/Alexdata
med/DS-Engineer-Test_Task](https://github.com/Alexdata
med/DS-Engineer-Test_Task)

Environment Constraints:
> Python 3.11+
> PyTorch
> Scikit-learn

Project Overview



Strict modular separation of concerns. Shared dependencies handled at the repository root.

Task 1 Requirements: Counting Islands

Input:

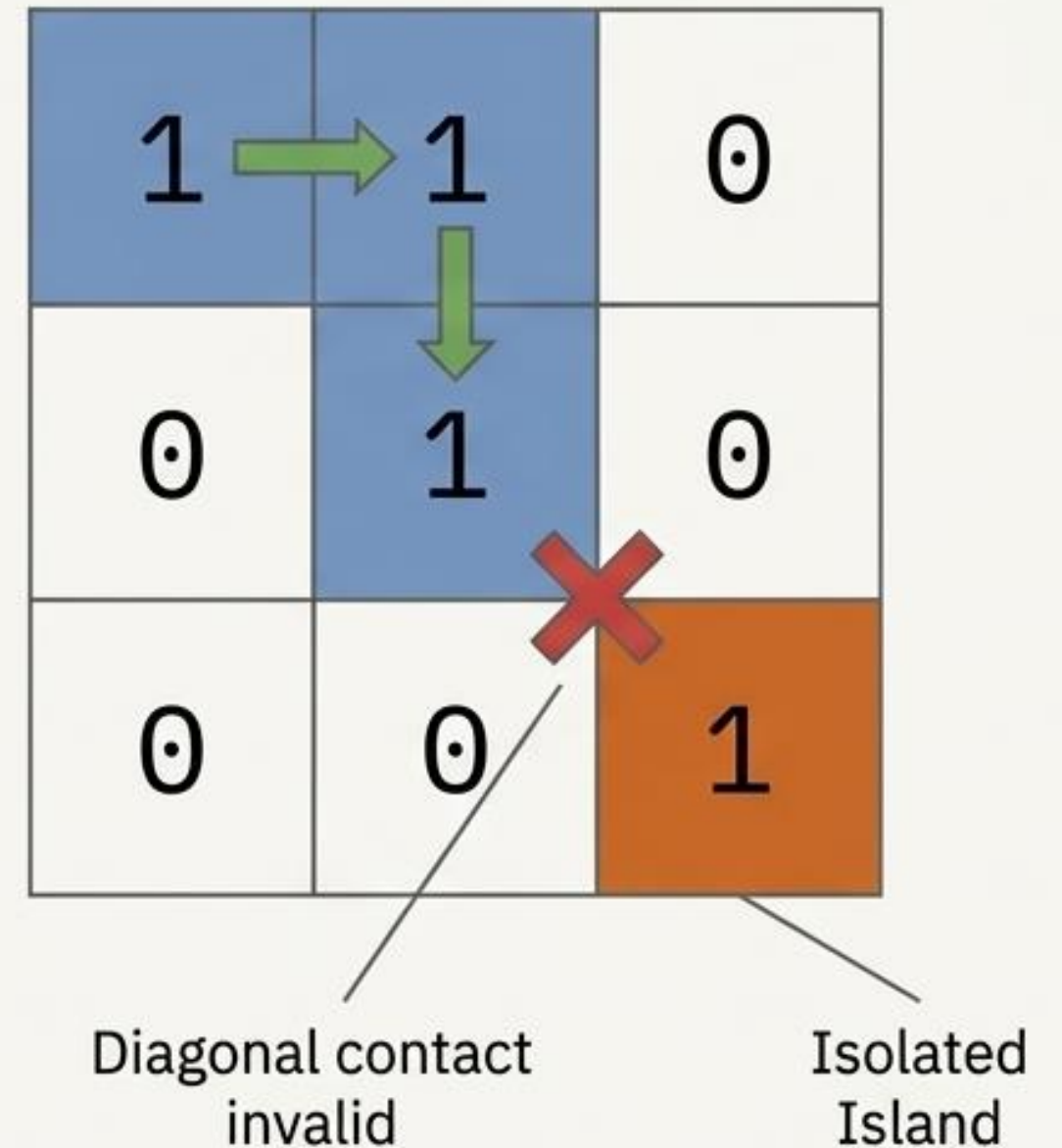
- Dimensions M N followed by an $M \times N$ matrix.
- Binary values: 1 (Land), 0 (Ocean).

Output:

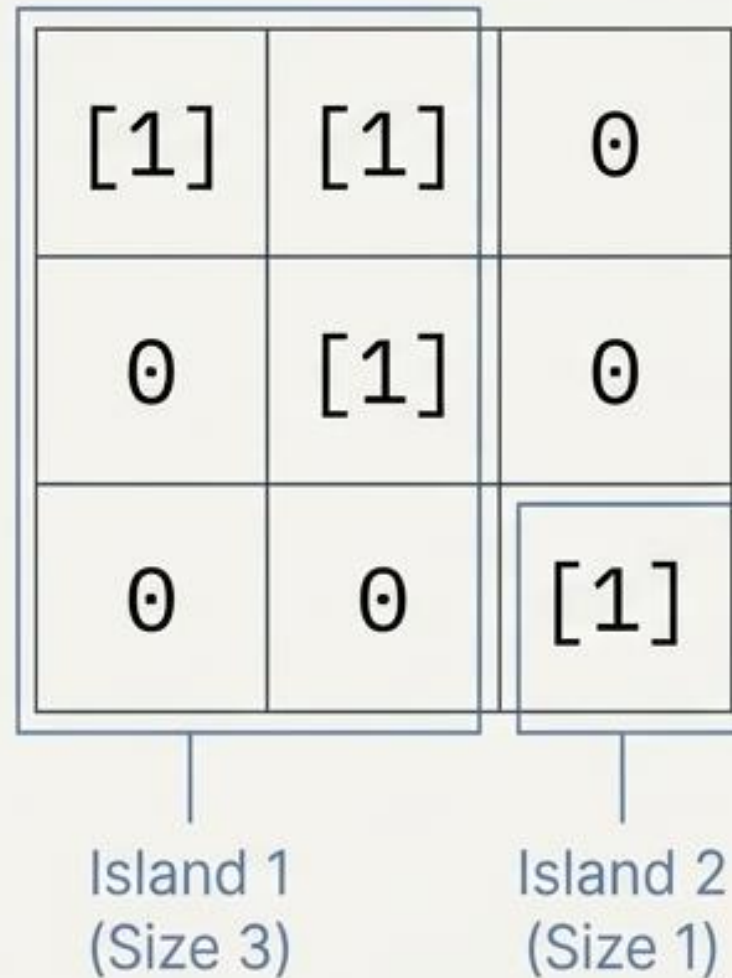
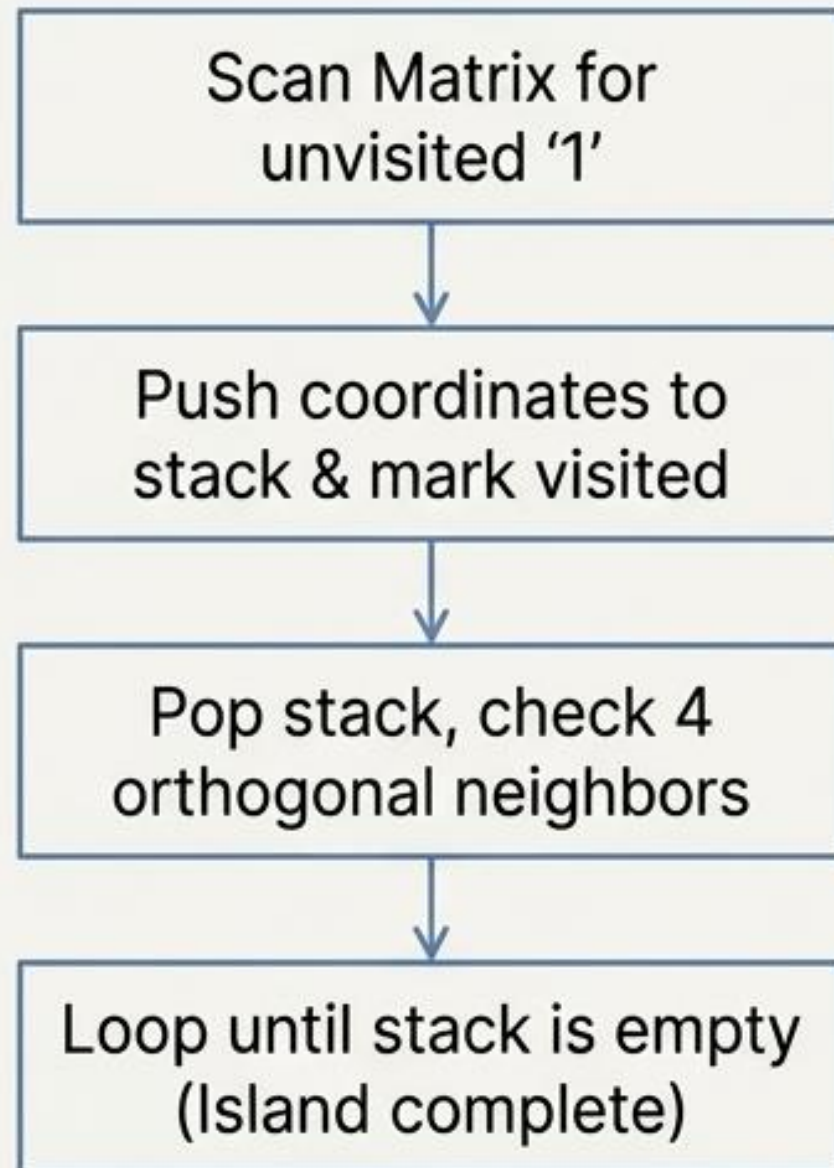
- A single integer representing total distinct islands.

Connectivity Rule:

- Strict 4-direction orthogonal connectivity (Up, Down, Left, Right).
- Diagonal contact does NOT connect land masses.



Task 1: Iterative DFS Implementation



Algorithm: Iterative Depth-First Search (DFS)

Complexity:

Time: $O(MN)$

Space: $O(MN)$

Engineering Trade-offs:

- Iteration avoids Python recursion depth limits on large grids.
- Separate "visited" set preserves the original input matrix integrity.

Task 1: Files and Verification

File Artifacts

```
task1_islands/  
├── __init__.py      # Package marker  
├── example.txt      # Specification input example  
├── solution.py      # Core algorithm & CLI logic  
└── test_solution.py # Automated unit tests
```

Execution & Verification

```
$ python task1_islands/solution.py < task1_islands/example.txt  
Matched supplied examples exactly: Outputs [2, 3, 2].
```

Verification Status

- Automated checks pass for empty grids, 2,000-cell islands, and invalid dimensions.

Task 2 Requirements: Tabular Regression

`train.csv`

90,000 labeled rows.

53 anonymized continuous features.

Target metric: Root Mean Square Error (RMSE).

`hidden_test.csv`

10,000 hidden-test rows.

True targets fundamentally unavailable.

Required Deliverables:



Executable training
& inference scripts



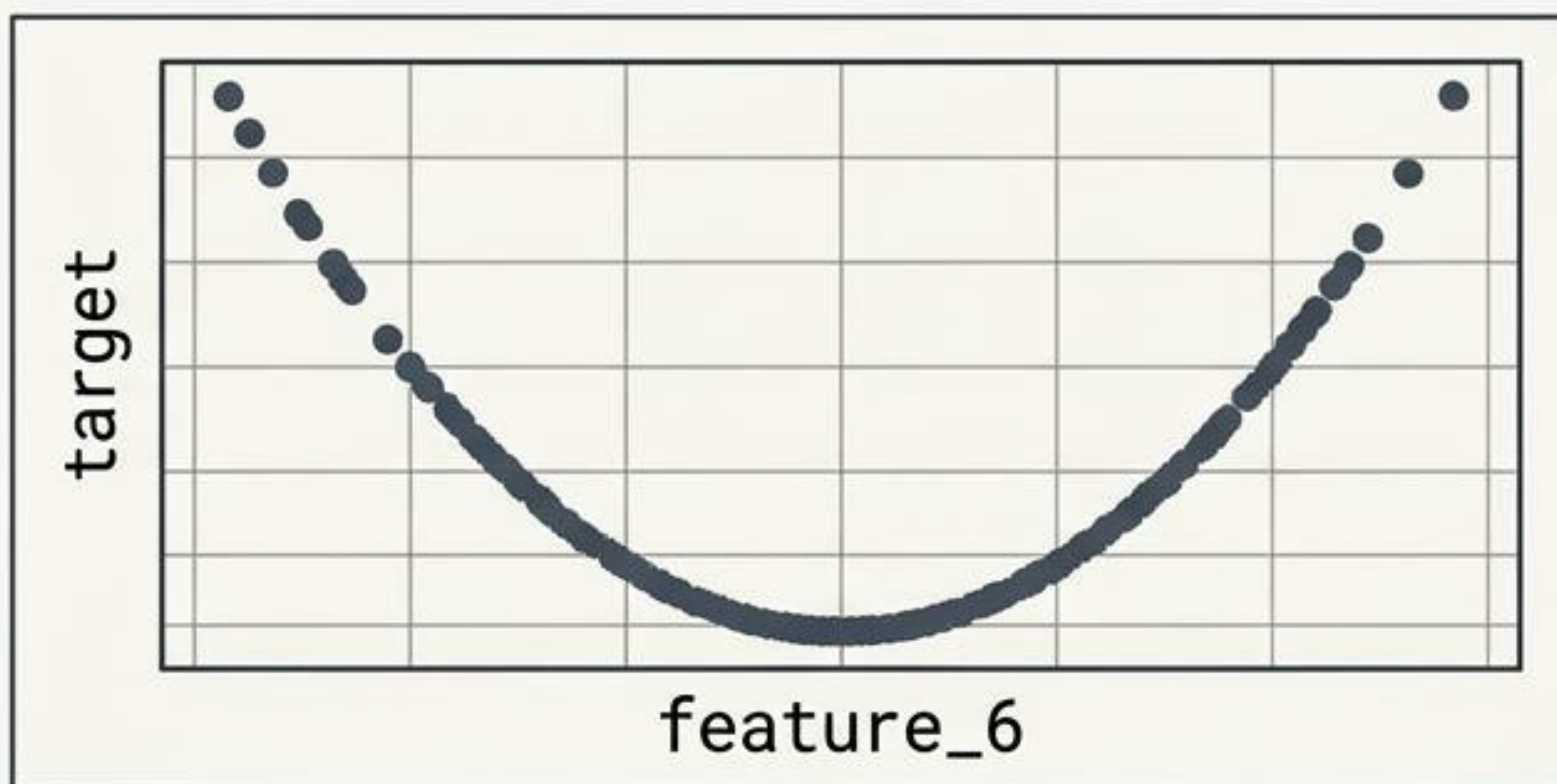
Final predictions
file (10,000 rows)



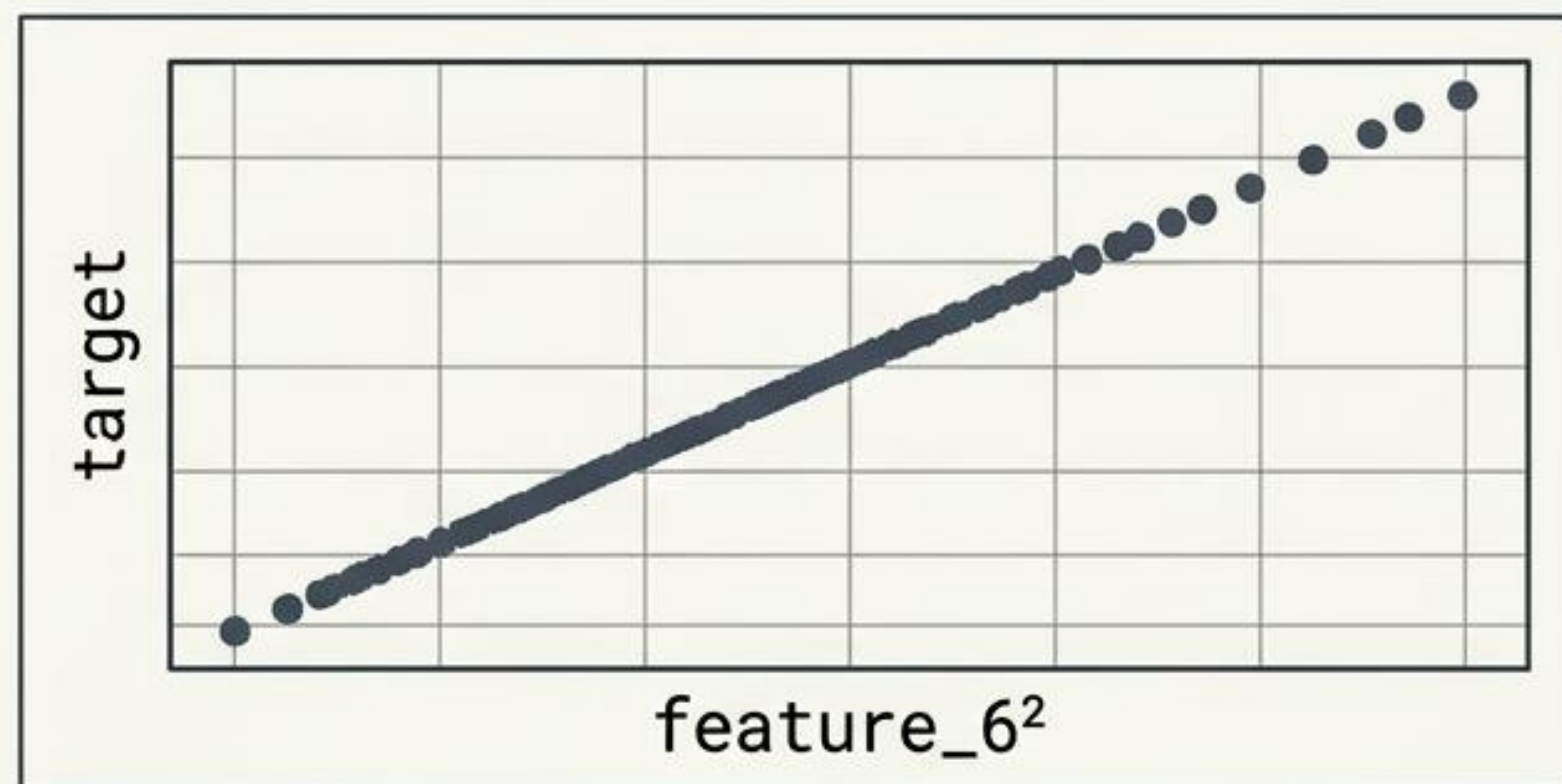
Executed EDA
Jupyter notebook

Task 2 EDA: Uncovering Non-Linearity

Insight: Weak linear correlation can hide a perfect quadratic relationship.



Standard linear correlation shows almost no signal.



Squaring the feature reveals a strictly linear, positive trend.

$$\text{target} \approx \text{feature_6}^2 + \text{feature_7}$$

Relationship observed within numerical precision during EDA.

Task 2: Forward Selection Architecture



Automated Discovery

The training algorithm discovers the selected feature indices automatically. They are NOT hardcoded.

Why this approach?

More complex models are unnecessary. Once squared features are added, the relationship is linear in its coefficients and can be solved optimally via least squares.

Task 2: Evaluation and Limitations

Seed 42

Train (60%)

Validation (20%)

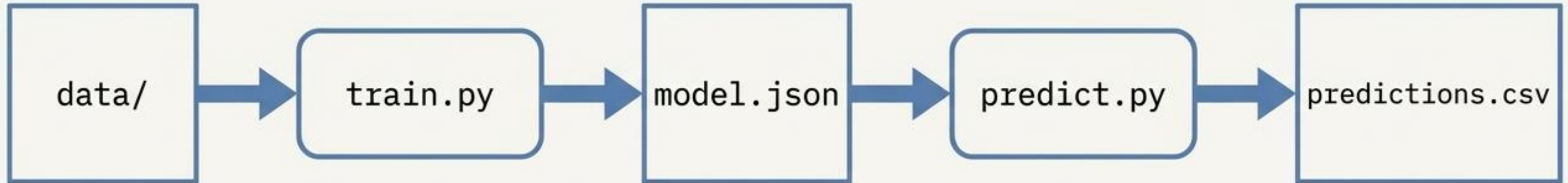
Holdout (20%)

Model	Partition	RMSE
Mean baseline	Validation	28.9164
Linear regression	Validation	28.9224
Two-term regression	Holdout	$\sim 3.20 \times 10^{-13}$

Methodological Limitations

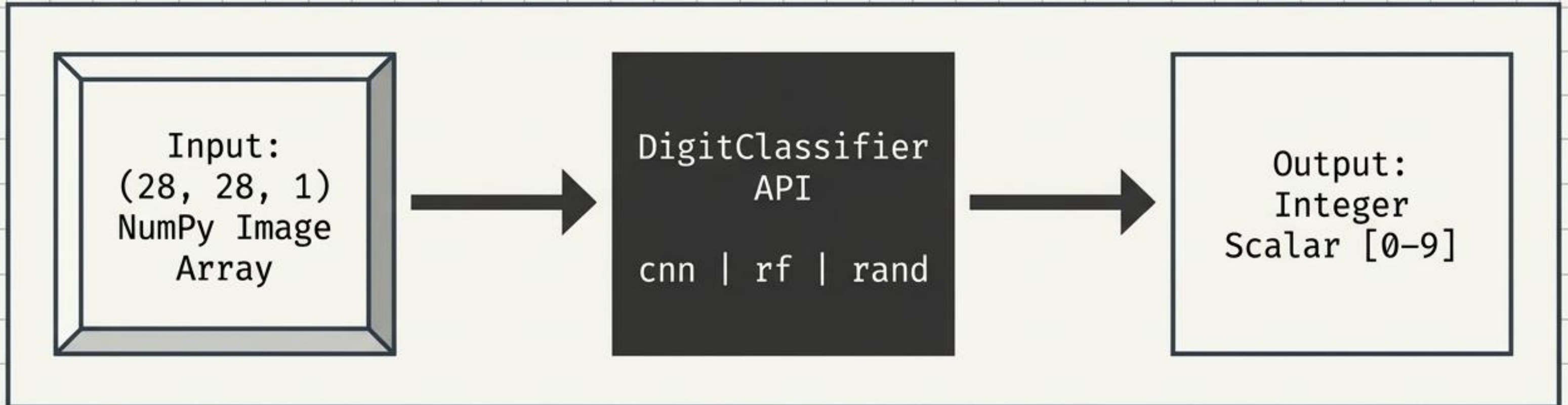
- Data Leakage: Prior full-dataset inspection suggested the hypothesis; the holdout was not fully blind to the discovery process.
- Unknown Hidden-Test RMSE: True performance on the hidden test set is fundamentally unavailable because labels are missing.
- Split Assumption: A random split assumes independent rows.

Task 2: Files and Execution



Core Architecture	Artifact Integrity
- regression.py: Shared training and inference logic. - notebooks/eda.ipynb: Executed exploratory analysis. - tests & reports: Validation and environment logging.	- Serialized cleanly into numeric JSON, avoiding Python pickle security issues. - Output contains exactly 10,000 predictions. - Predictions perfectly preserve the original row order.

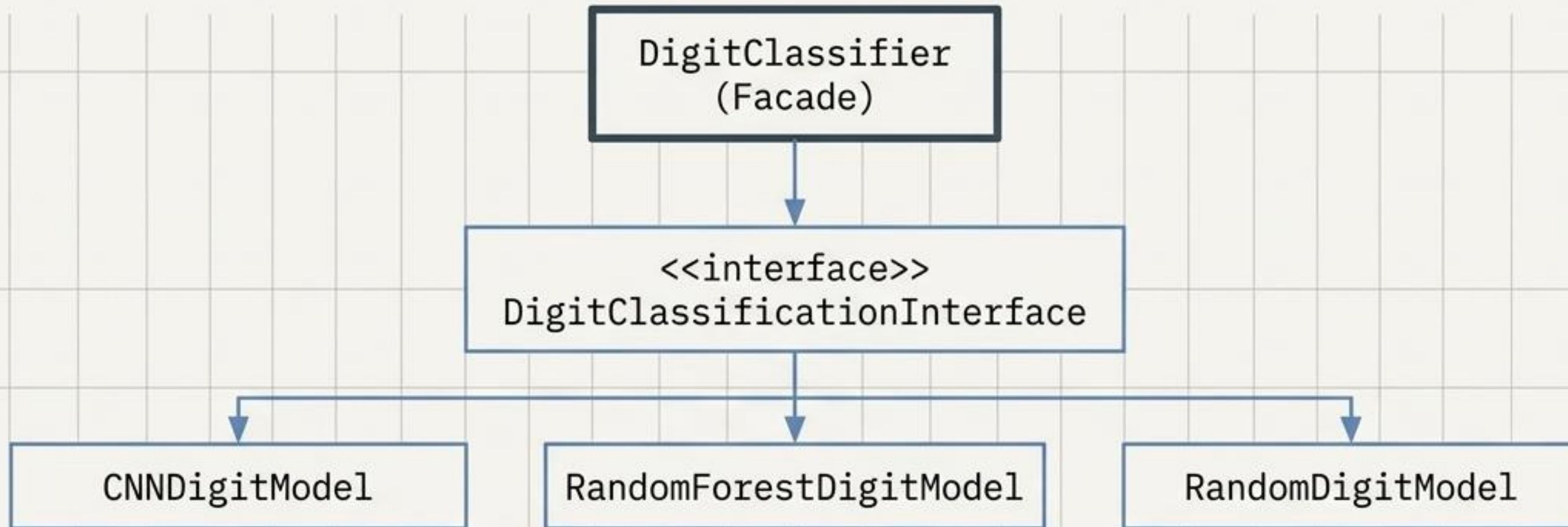
Task 3 Requirements: Unified Inference API



Constraints & Out of Scope

- Unified single entry point for all three distinct machine learning backends.
- MNIST accuracy is strictly outside the scope of this architecture task.
- Model training is not implemented (explicitly raises `NotImplementedError`).

Task 3: OOP Strategy and Facade



Algorithm	Input Adaptation Format	Internal Normalization
CNN	PyTorch Tensor: (1, 1, 28, 28)	[0, 255] -> [0, 1]
RF	Flattened Vector: length 784	
Rand	Central Crop: (10, 10)	

Task 3: Files, Verification, and Limitations

File Modules

task3_mnist/

- classifier.py # Facade and adaptation
- interfaces.py # Abstract base class
-  models/
 - cnn.py # PyTorch backend
 - random_forest.py # Scikit-Learn backend
 - random_model.py # NumPy backend
-  tests/

Backend Injection & Testing Scope

```
classifier = DigitClassifier('rf',  
                             backend=fitted_rf)
```

- **Backend Injection:** CNN and RF require an explicitly supplied backend at initialization.
- **Missing Weights:** Trained MNIST weights are purposefully NOT included in the repository.
- **Verification Scope:** Unit tests validate the inference format contracts, input adaptation logic, and exceptions—NOT classification accuracy.

Project Verification and Closing

Verification Metrics

 **Passing** GitHub Actions CI/CD

Local Verification Baseline:

- 9 fully passing unit tests across all tasks.
- Strictly reproducible tabular regression predictions.
- 6 error-free notebook cell executions.

Core Engineering Decisions Summary

Task 1: Iterative graph traversal (DFS) to prevent recursion depth failures.

Task 2: Automated forward feature selection to mathematically recover hidden signals.

Task 3: Strategy design pattern with a Facade for decoupled input adaptation.

Repository: https://github.com/Alexdatamed/DS-Engineer-Test_Task